

cURL

Tester, déboguer et automatiser les appels HTTP en ligne de commande

cURL	HTTP	REST	API
------	------	------	-----

Headers	Auth	Webhooks	Débogage
---------	------	----------	----------

Parcours de formation — Développement d'Applications Mobiles

Djiguitech · Social Seller · Tous niveaux

■ Compétences visées

- ✓ Comprendre le protocole HTTP et ses méthodes (GET, POST, PUT, PATCH, DELETE)
- ✓ Maîtriser la syntaxe cURL pour tester n'importe quelle API REST
- ✓ Envoyer des headers, des tokens d'authentification et des payloads JSON
- ✓ Déboguer les réponses HTTP : codes de statut, headers, body
- ✓ Tester les webhooks et les routes sécurisées d'une API Express
- ✓ Automatiser des séquences de requêtes avec des variables shell

Table des matières

Chapitre 1 — HTTP revisité — le protocole sous-jacent

- 1.1 Requête et réponse
- 1.2 Méthodes HTTP
- 1.3 Codes de statut
- 1.4 Headers importants

Chapitre 2 — cURL — syntaxe fondamentale

- 2.1 Installation et première requête
- 2.2 Options essentielles
- 2.3 Verbose mode — voir tout
- 2.4 Sauvegarder et rejouer

Chapitre 3 — Authentification et headers

- 3.1 Bearer token
- 3.2 Basic auth
- 3.3 Headers custom
- 3.4 Cookies

Chapitre 4 — Envoyer des données

- 4.1 JSON body avec POST
- 4.2 PATCH et PUT
- 4.3 Form data
- 4.4 Upload de fichier

Chapitre 5 — Cas pratiques — Social Seller

- 5.1 Tester le health check
- 5.2 Tester les webhooks
- 5.3 Tester les routes protégées

5.4 Scripts de test automatisés

HTTP revisité — le protocole sous-jacent

1.1 Requête et réponse

HTTP (HyperText Transfer Protocol) est le protocole de communication du web. Chaque interaction se déroule en deux temps : le client envoie une requête, le serveur répond. Une requête HTTP contient une ligne de départ (méthode + URL + version), des headers (métadonnées), et optionnellement un body (données). La réponse contient un code de statut, des headers, et un body.

cURL (Client URL) est un outil en ligne de commande qui permet de forger des requêtes HTTP manuellement. C'est l'outil indispensable du développeur backend pour tester une API sans avoir besoin d'une interface graphique, d'une app mobile, ou de Postman. Comprendre cURL, c'est comprendre HTTP.

```
# Anatomie d'une requête HTTP
GET /health HTTP/1.1

Host: social-seller-production.up.railway.app
Authorization: Bearer eyJhbGci...
Content-Type: application/json

# Anatomie d'une réponse HTTP
HTTP/1.1 200 OK
Content-Type: application/json
X-Request-Id: abc123

{ "status": "ok" }
```

1.2 Méthodes HTTP

Les 5 méthodes HTTP REST fondamentales

Méthode	Usage	Body ?	Idempotent ?
GET	Lire une ressource	Non	Oui
POST	Créer une ressource	Oui	Non
PUT	Remplacer une ressource entière	Oui	Oui
PATCH	Modifier partiellement	Oui	Non
DELETE	Supprimer une ressource	Non	Oui

L'idempotence signifie qu'appeler la même requête plusieurs fois produit le même résultat. GET /orders/123 retourne toujours la même commande. DELETE /orders/123 supprime la commande — si on l'appelle une deuxième fois, la commande n'existe plus et on reçoit 404. POST /orders crée une nouvelle commande à chaque appel — pas idempotent.

1.3 Codes de statut

Codes de statut HTTP les plus courants

Code	Signification	Cas typique
200 OK	Succès	GET, PATCH réussi
201 Created	Ressource créée	POST réussi
204 No Content	Succès sans body	DELETE réussi
400 Bad Request	Données invalides	Validation Zod échouée
401 Unauthorized	Non authentifié	Token manquant ou expiré
403 Forbidden	Non autorisé	Rôle insuffisant
404 Not Found	Ressource introuvable	ID inconnu ou cross-tenant
409 Conflict	Conflit d'état	Transition de statut invalide
429 Too Many Requests	Rate limit atteint	Trop de requêtes
500 Internal Server Error	Erreur serveur	Bug non géré

1.4 Headers importants

Les headers HTTP transmettent des métadonnées sur la requête et la réponse. Certains sont standardisés, d'autres sont custom. Les headers les plus importants pour une API REST sont : Content-Type (format des données), Authorization (token d'auth), Accept (format attendu en réponse), et X-Request-Id (identifiant de requête pour le debugging).

cURL — syntaxe fondamentale

2.1 Installation et première requête

```
# cURL est préinstallé sur macOS et Linux

curl --version

# curl 8.x.x (x86_64-apple-darwin) ...

# Requête GET simple

curl https://social-seller-production.up.railway.app/health

# Réponse : {"status":"ok"}

# Formater le JSON en sortie (nécessite jq)

brew install jq

curl https://api.example.com/products | jq .
```

2.2 Options essentielles

Options cURL les plus utilisées

Option	Forme courte	Description
--request	-X	Méthode HTTP (GET par défaut)
--header	-H	Ajouter un header
--data	-d	Body de la requête
--silent	-s	Supprimer la barre de progression
--include	-i	Afficher les headers de réponse
--verbose	-v	Afficher tout (requête + réponse)
--output	-o	Sauvegarder la réponse dans un fichier
--location	-L	Suivre les redirections

2.3 Verbose mode — voir tout

Le mode verbose (-v) est l'outil de débogage ultime de cURL. Il affiche la requête complète envoyée (headers inclus), la réponse complète reçue (headers + body), et les détails de la connexion TLS. Les lignes commençant par > sont la requête, celles commençant par < sont la réponse.

```
curl -v https://social-seller-production.up.railway.app/health
```

```
# Sortie :
```

```
# * Trying 34.x.x.x:443...
```

```
# * Connected to social-seller-production.up.railway.app
```

```
# * SSL certificate verified
```

```
> GET /health HTTP/2
```

```
> Host: social-seller-production.up.railway.app
```

```
> User-Agent: curl/8.4.0
```

```
> Accept: */*
```

```
>
```

```
< HTTP/2 200
```

```
< content-type: application/json
```

```
< x-request-id: abc123
```

```
<
```

```
{ "status": "ok" }
```


Authentification et headers

3.1 Bearer token

La grande majorité des APIs modernes utilisent l'authentification par Bearer token. Le token JWT est envoyé dans le header Authorization avec le préfixe Bearer. En shell, il est pratique de stocker le token dans une variable pour ne pas le répéter dans chaque commande.

```
# Stocker le token dans une variable shell
TOKEN="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."

# L'utiliser dans une requête
curl -s https://api.example.com/products \
-H "Authorization: Bearer $TOKEN"

# Avec plusieurs headers
curl -s https://api.example.com/products \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-H "Accept: application/json"

# Vérifier le code de statut seulement
curl -s -o /dev/null -w "%{http_code}" \
-H "Authorization: Bearer $TOKEN" \
https://api.example.com/products

# Affiche : 200
```

3.2 Tester sans token et avec token invalide

```
# Sans token → doit retourner 401
curl -s -o /dev/null -w '%{http_code}' \
https://social-seller-production.up.railway.app/products

# 401

# Avec token invalide → doit retourner 401
curl -s https://social-seller-production.up.railway.app/products \
-H "Authorization: Bearer token_bidon"
```

```
# {"error":{"code":"unauthorized",...}}
```

```
# Avec token valide → 200
```

```
curl -s https://social-seller-production.up.railway.app/products \
```

```
-H "Authorization: Bearer $TOKEN"
```

```
# [{"id":"...", "name":"..."}]
```

Envoyer des données

4.1 JSON body avec POST

Pour envoyer des données JSON, deux éléments sont indispensables : le header Content-Type: application/json pour indiquer au serveur le format des données, et l'option -d (ou --data) avec les données JSON entre guillemets simples. Sur macOS, les guillemets simples permettent d'éviter l'interprétation des guillemets doubles par le shell.

```
# Créer un produit (POST avec JSON)

curl -s -X POST https://social-seller-production.up.railway.app/products \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "name": "Boubou brodé",
  "price": 15000,
  "stock_quantity": 10
}'

# Modifier partiellement (PATCH)

curl -s -X PATCH https://social-seller-production.up.railway.app/products/abc-123 \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"stock_quantity": 5}'

# Supprimer (DELETE)

curl -s -X DELETE https://social-seller-production.up.railway.app/products/abc-123 \
-H "Authorization: Bearer $TOKEN"
```

4.2 Tester la validation des inputs

cURL est idéal pour tester que le serveur valide correctement les inputs. On envoie délibérément des données invalides et on vérifie que la réponse est 400 avec un message d'erreur clair. C'est un test de sécurité fondamental.

```
# Prix négatif → doit retourner 400

curl -s -X POST https://api.example.com/products \
```

```
-H "Authorization: Bearer $TOKEN" \  
-H "Content-Type: application/json" \  
-d '{"name": "Test", "price": -500}'  
# {"error":{"code":"validation_error",...}}  
  
# Champ obligatoire manquant → 400  
curl -s -X POST https://api.example.com/products \  
-H "Authorization: Bearer $TOKEN" \  
-H "Content-Type: application/json" \  
-d '{"price": 5000}'  
# {"error":{"details":{"name":["Required"]}}}
```

Cas pratiques — Social Seller

5.1 Tester les webhooks

Les webhooks ont deux endpoints à tester : la vérification (GET avec `hub.challenge`) et la réception de messages (POST avec payload signé). cURL permet de simuler les deux sans dépendre d'un vrai message entrant.

```
# Vérification webhook WhatsApp (Meta envoie un GET)

curl -s "https://social-seller-production.up.railway.app/webhooks/whatsapp\
?hub.mode=subscribe\
&hub.verify_token=ss_wh_verify_2024\
&hub.challenge=test123"

# Réponse attendue : test123

# Token incorrect → 403

curl -s -o /dev/null -w "%{http_code}" \
"https://social-seller-production.up.railway.app/webhooks/whatsapp\
?hub.mode=subscribe\
&hub.verify_token=MAUVAIS_TOKEN\
&hub.challenge=test123"

# 403

# Simuler un message entrant WhatsApp (POST)

curl -s -X POST https://social-seller-production.up.railway.app/webhooks/whatsapp \
-H "Content-Type: application/json" \
-H "X-Hub-Signature-256: sha256=SIGNATURE" \
-d '{"object":"whatsapp_business_account","entry":[]}'
```

5.2 Scripts de test automatisés

Les scripts shell combinant cURL et des variables permettent d'automatiser une suite de tests sans framework. C'est utile pour des smoke tests rapides après un déploiement en production.

Script de smoke test automatisé

```
#!/bin/bash

# smoke_test.sh – tests rapides après déploiement
```

```

BASE_URL="https://social-seller-production.up.railway.app"
TOKEN="$1" # Passer le token en argument : ./smoke_test.sh eyJ...

# Fonction utilitaire
check() {
    local desc="$1" expected="$2" actual="$3"
    if [ "$actual" = "$expected" ]; then
        echo "✓ $desc"
    else
        echo "✗ $desc (attendu: $expected, reçu: $actual)"
    fi
}

# Health check
STATUS=$(curl -s -o /dev/null -w "%{http_code}" $BASE_URL/health)
check "Health check" "200" "$STATUS"

# Auth sans token
STATUS=$(curl -s -o /dev/null -w "%{http_code}" $BASE_URL/products)
check "Auth requis sans token" "401" "$STATUS"

# Auth avec token valide
STATUS=$(curl -s -o /dev/null -w "%{http_code}" \
-H "Authorization: Bearer $TOKEN" $BASE_URL/products)
check "Auth valide" "200" "$STATUS"

# Webhook verify
BODY=$(curl -s "$BASE_URL/webhooks/whatsapp?hub.mode=subscribe\
&hub.verify_token=ss_wh_verify_2024&hub.challenge=ping")
check "Webhook verify" "ping" "$BODY"

echo "Smoke tests terminés."

```

5.3 Astuces et raccourcis

```

# Afficher uniquement le code de statut
curl -s -o /dev/null -w "%{http_code}" URL

# Afficher statut + body
curl -s -w "\n\nHTTP Status: %{http_code}\n" URL

# Suivre les redirections automatiquement

```

```

curl -sL URL

# Timeout de 5 secondes
curl -s --max-time 5 URL

# Ignorer les erreurs SSL (dev uniquement !)
curl -sk URL

# Sauvegarder la réponse dans un fichier
curl -s URL -o response.json && jq . response.json

# Envoyer plusieurs requêtes en parallèle (avec &)
for i in 1 2 3 4 5; do
  curl -s URL -H "Authorization: Bearer $TOKEN" &
done
wait

# Utile pour tester le rate limiting

```

■ Bonne pratique

Créer des alias shell pour les commandes cURL fréquentes. Par exemple : `alias api='curl -s -H "Content-Type: application/json" -H "Authorization: Bearer $TOKEN"'`. Tu peux ensuite écrire : `api https://api.example.com/products | jq .`

■ À retenir

- ✓ HTTP = requête (méthode + URL + headers + body) + réponse (statut + headers + body)
- ✓ `curl -s` supprime la barre de progression | `curl -v` affiche tout
- ✓ Toujours spécifier `Content-Type: application/json` avec `-d` pour les APIs REST
- ✓ Stocker les tokens dans des variables shell pour éviter la répétition
- ✓ `-w '%{http_code}' + -o /dev/null` = afficher uniquement le code de statut
- ✓ Les scripts shell cURL automatisent les smoke tests après chaque déploiement

— ■ Exercices

Exercice 1 — Tester son API complète

Écrire une suite de tests cURL pour toutes les routes de l'API Social Seller.

- Health check : GET /health → 200
- Sans token : GET /products → 401
- Token invalide : GET /products avec 'Bearer bidon' → 401
- Webhook verify WhatsApp : GET avec bon token → challenge retourné
- Webhook verify mauvais token → 403

Exercice 2 — Script de smoke test

Créer un script smoke_test.sh réutilisable.

- Prendre le BASE_URL et TOKEN en arguments
- Tester au moins 5 routes différentes
- Afficher ✓ ou ✗ pour chaque test
- Retourner un code de sortie non-zéro si au moins un test échoue

Exercice 3 — Tester la validation

Vérifier que les validations Zod fonctionnent via cURL.

- POST /products sans le champ 'name' → 400
- POST /products avec price négatif → 400
- POST /products avec tous les champs valides → 201
- Vérifier que le body d'erreur contient les détails de validation

Bibliographie & Ressources

Documentation

- curl.se/docs — documentation officielle cURL
- curl.se/book — Everything curl (livre gratuit)
- httpie.io — alternative moderne à cURL (plus lisible)

Outils complémentaires

- stedolan.github.io/jq — jq, processeur JSON en ligne de commande
- hurl.dev — tester des APIs HTTP avec un format déclaratif
- requestbin.com — inspecter les requêtes entrantes (webhooks)